Microcontrollers Project: Simon Says Game

Bachelor's Degree in Electronic, Robotics and Mechatronics Engineering

Author: Jaime Esparragoso Vides

Date: January 30, 2026

Universidad de Sevilla

Contents

1 Introduction

This report summarizes the development of our project for the course: we implement the classic "Simon Says" game using the MSP430 microcontroller and the BoosterPack MkII. The idea is to meet all the points proposed in the assignment, as specified in the example PDF.

2 Hardware and Peripheral Configuration

For this project we used libraries for the display, so that we could show whatever we wanted on the LCD screen. The rest of the functionality we implemented ourselves in the code:

- **Joystick (ADC10):** We needed to continuously read the stick's position. We configured the ADC to read channels A0 and A3. The trickiest part here was calibrating the values: the joystick needs tolerance margins so the game doesn't detect phantom movements.
- **Timers:** We used two:
  - *Timer0:* Our main clock. It gives us an interrupt every 25 ms. This lets us time the rounds without using `delay()`, which is key because it means the microcontroller keeps detecting button presses while it "waits".
  - *Timer1:* Dedicated exclusively to audio. We use it in compare mode to output a PWM signal to the buzzer. By changing this timer's frequency on the fly we get the different musical notes.
- **Interrupts:** For buttons S1, S2, and the joystick button, we use interrupts, which drive the behavior carried out by these inputs.
- **LCD Screen:** We use SPI communication, all handled inside the library. It is the most resource-intensive part, so we had to optimize the drawing code so it wouldn't slow down the game logic.

3 Code Structure

The structure we followed for our code is as follows:

3.1 The Main Loop

The core of the program (`main`) consists of an infinite loop (`while(1)`) that puts the microcontroller into low-power mode (`LPM0`) on every iteration. The system only "wakes up" through interrupts. The most important interrupt is **Timer0**'s, configured to fire every 25 milliseconds. This interrupt raises a global flag called `tick`.

- If `tick == 0`: The processor sleeps, saving power.
- If `tick == 1`: The main loop runs one cycle of game logic, updates counters, and goes back to sleep.

3.2 State Machine (FSM)

To manage the game logic, we implemented a State Machine controlled by the `estado` variable. The defined states (`estados_t`) are:

1. **BIENVENIDA:** Initial idle state. Waits for the player to press "Start". Plays a looping melody and checks whether "Symbol Mode" is activated.

2. **MENSAJE_RONDA:** Transition state that clears the screen and announces the current round number to the player.
3. **MAQUINA:** The system steps through the `secuencia` array and lights up the corresponding colors/symbols.
4. **TURNO_JUGADOR:** The system waits for input from the ADC (joystick). It compares the user's input against the stored sequence. If there is an error or the time runs out (*timeout*), we move to `FIN`. If the sequence is completed, we move to `VICTORIA`.
5. **VICTORIA:** We show an animation and success music after clearing a round.
6. **VICTORIA_FINAL:** State reached after clearing all 32 rounds. Shows a star animation and increases the difficulty.
7. **FIN:** "Game Over" screen; shows the final score and allows restarting.

4 Detailed Description of Functions and Constants

A brief expansion on some of the constants and functions declared:

4.1 Definitions and Constants (`#define`)

Macros were mainly used for two purposes: music and graphics.

- **Musical Notes:** We defined the note frequencies (e.g. `#define DO 262`, `#define LA 440`). This makes writing melodies in the code much simpler and more intuitive.
- **Hexadecimal Colors:** The graphics library uses 32-bit values. We defined constants such as `GRAPHICS_COLOR_BRIGHT_GREEN` so we could create our own custom colors.
- **Animation Parameters:** Constants such as `N_VICTF` or `N_ESTRELLAS` control the length of the melodies and the number of little stars on screen.

4.2 Critical Global Variables

Because Interrupt Service Routines (ISRs) cannot return values, we use `volatile` global variables for communication between the interrupts and the main loop:

- `volatile unsigned char tick`: This variable is the system's main clock.
- `volatile unsigned char start`: Indicates whether the joystick button has been pressed.
- `lfsr`: 16-bit shift register for the pseudorandom number generator.

4.3 Initial Functions

These functions isolate the game logic from the MSP430-specific registers, making the code more portable and readable.

`Set_Clk(char VEL)` Configures the timer to run at 1, 8, or 16 MHz. For this game, we set it to 16 MHz.

`inicia_ADC(char canales)` and `lee_ch(char canal)` Configure the ADC and read its value.

`init_buzzer()`, `suena_hz()` and `apaga_sonido()` Control Timer1.

- `suena_hz` calculates the value of the `TA1CCR0` register based on the desired frequency, to generate a square-wave (50% PWM) signal.
- `apaga_sonido` stops the timer to silence the system.

4.4 Logic and Algorithm Functions

`lfsr_siguiente()` and `aleatorio_1_4()` Implement the pseudorandom number generator (Requirement 2.3). We use an LFSR (Linear Feedback Shift Register) algorithm that generates a random sequence to play with.

`dibuja_estrella()` A graphics helper function that draws shapes at random (x,y) coordinates for the final-victory animation.

5 Requirements Implementation

Below, we detail how we translated each point of the assignment into our C code:

5.1 General Features and Accessibility (Items 1.x)

- **Colors and Sounds (1.1):** We defined the frequencies (`#define DO 262`, etc.) and the colors in arrays (`color_alto[]` and `color_bajo[]`). This lets us work with them easily.
- **Colorblind Mode (1.2):** In the start routine, we also check whether the player is holding the joystick to the left (`ejex < 100`). If so, we set a flag `modo_simbolos = 1`. From that point on, all drawing logic splits into two paths: it either draws colors, or it draws the characters #, @, $ and % on a black background.
- **Timing (1.3):** The game's timings are not fixed, but variable. We compute `T` and `T4` based on a `tiempo_base` variable (1000 ms at the start). The state machine uses counters (`tms`) that increment on every timer tick.
- **Visual Feedback (1.4):** To simulate "lighting up", we redraw the same rectangle but change the color from `color_bajo` to `color_alto`.

5.2 Random Generation (Items 2.x)

- **Random Sequence (2.3):** To comply with the restriction, we do not use `stdlib.h` for randomness. We use a 16-bit LFSR (Linear Feedback Shift Register) in the `lfsr_siguiente()` function. Through XOR operations and bit shifts, we obtain a pseudorandom sequence.
- **Human Seed (2.2):** To make every playthrough unique, the initial seed (`lfsr`) is loaded with the value of `contador_ticks` when the player presses the Start button.
- **Cumulative Sequence (2.5):** We use a single `secuencia[32]` array. In round 1, the machine reads up to index 0; in round 2, up to index 1, and so on. We do not generate a new sequence every round; instead we reuse the initial sequence of 32 terms.

5.3 Interface and Game Flow (Items 3, 4 and 5)

- **Game States:** We split the flow into clear states:
  - `BIENVENIDA`: The opening melody plays and it waits for the button.
  - `MENSAJE_RONDA`: Shows the current round number.
  - `MAQUINA`: Plays back the sequence using the `secuencia` array and the timer.
  - `TURNO_JUGADOR`: Reads the joystick. If the player takes longer than `2*T` (controlled by `tms >= T2`), it jumps to `FIN`.

- **Score (5.7):** We compute the score step by step. Every correct answer increments the `puntuacion` variable. This means that if you fail partway through a round, you still keep the points for the steps you got right.
- **Hard Mode (5.5):** If the player clears all 32 rounds, the game enters `VICTORIA_FINAL`. If Start is pressed again, we restart the game by setting `modo_rapido = 1`, which halves the base time, making the game twice as fast.

5.4 High Scores and Improvements

We were not able to make progress on this part due to the board's limited resources, since at a certain point it started throwing errors and displaying random colors (video attached). We do have the code for how we implemented the high-score screen to save them, even though it ended up being on an external one.

6 Source Code

Below is the complete code implemented for the project:

```
#include <msp430.h>
#include <stdio.h>
#include <stdint.h>

#include "grlib.h"
#include "Crystalfontz128x128_ST7735.h"
#include "HAL_MSP430G2_Crystalfontz128x128_ST7735.h"

int i=0;

// Notas musicales y sus frecuencias
#define DO  262
#define MI  330
#define SOL 392
#define SI  494
#define DO2 523

// Notas extra para la victoria final
#define RE   294
#define FA   349
#define LA   440
#define RE2 587
#define MI2 659
#define SOL2 784

#define GRAPHICS_COLOR_DARK_YELLOW 0x008B5400
#define GRAPHICS_COLOR_BRIGHT_GREEN   0x007FFF00   // verde muy vivo

Graphics_Context g_sContext;

// Modo accesible: 0 = colores, 1 = simbolos
unsigned char modo_simbolos = 0;

// Flags de despertar
volatile unsigned char tick = 0;
volatile unsigned char start = 0;
volatile unsigned long contador_ticks = 0;

// Boton 1 (P1.1)
volatile unsigned char boton1 = 0;

```

```
42 // Joystick inicial
43 volatile unsigned int ejex = 512;
44 volatile unsigned int ejey = 512;
45
46 // Funcion reloj
47 void Set_Clk(char VEL){
48     BCSCTL2 = SELM_0 | DIVM_0 | DIVS_0;
49     switch(VEL){
50     case 16:
51         if (CALBC1_16MHZ != 0xFF) {
52             __delay_cycles(100000);
53             DCOCTL = 0x00;
54             BCSCTL1 = CALBC1_16MHZ;
55             DCOCTL = CALDCO_16MHZ;
56         }
57         break;
58     case 8:
59         if (CALBC1_8MHZ != 0xFF) {
60             __delay_cycles(100000);
61             DCOCTL = 0x00;
62             BCSCTL1 = CALBC1_8MHZ;
63             DCOCTL = CALDCO_8MHZ;
64         }
65         break;
66     default:
67         if (CALBC1_1MHZ != 0xFF) {
68             DCOCTL = 0x00;
69             BCSCTL1 = CALBC1_1MHZ;
70             DCOCTL = CALDCO_1MHZ;
71         }
72         break;
73     }
74     BCSCTL1 |= XT2OFF | DIVA_0;
75     BCSCTL3 = XT2S_0 | LFXT1S_2 | XCAP_1;
76 }
77
78 // Funcion que inicializa el ADC
79 void inicia_ADC(char canales){
80     ADC10CTL0 &= ~ENC;
81     ADC10CTL0 = ADC10ON | ADC10SHT_3 | SREF_0 | ADC10IE;
82     ADC10CTL1 = CONSEQ_0 | ADC10SSEL_0 | ADC10DIV_0 | SHS_0 | INCH_0;
83     ADC10AE0 = canales;
84     ADC10CTL0 |= ENC;
85 }
86
87 // Funcion para leer el valor del ADC
88 int lee_ch(char canal){
89     ADC10CTL0 &= ~ENC;
90     ADC10CTL1 &= (0x0fff);
91     ADC10CTL1 |= canal<<12;
92     ADC10CTL0 |= ENC;
93     ADC10CTL0 |= ADC10SC;
94     LPM0;
95     return (ADC10MEM);
96 }
97
98 // Rutina de interrupcion ADC
99 #pragma vector=ADC10_VECTOR
100 __interrupt void ConvertidorAD(void){
101     LPM0_EXIT;
102 }
103
104 // Funcion Timer 25ms
```

```
105 void timer_tick(void){
106     TA0CTL = TASSEL_2 | ID_3 | MC_1;  // SMCLK/8 = 2MHz
107     TA0CCR0 = 49999;                  // 25ms
108     TA0CCTL0 = CCIE;
109 }
110 
111 // Rutina de interrupcion Timer
112 #pragma vector=TIMER0_A0_VECTOR
113 __interrupt void Interrupcion_T0(void){
114     tick = 1;
115     contador_ticks++;
116     LPM0_EXIT;
117 }
118 
119 // Funcion que inicializa el boton de start (boton del joystick)
120 void boton_start(void){
121     P2DIR &= ~BIT5;
122     P2REN |= BIT5;
123     P2OUT |= BIT5;
124     P2IFG &= ~BIT5;
125     P2IES |= BIT5;
126     P2IE  |= BIT5;
127 }
128 
129 // Funcion que inicializa el boton 1 (P1.1)
130 void boton_1(void){
131     P1DIR &= ~BIT1;
132     P1REN |= BIT1;
133     P1OUT |= BIT1;
134     P1IFG &= ~BIT1;
135     P1IES |= BIT1;
136     P1IE  |= BIT1;
137 }
138 
139 // Rutina de interrupcion Boton Joystick
140 #pragma vector=PORT2_VECTOR
141 __interrupt void Interrupcion_P2(void){
142     if(!(P2IN & BIT5)){
143         start = 1;
144     }
145     P2IFG &= ~BIT5;
146     LPM0_EXIT;
147 }
148 
149 // Rutina de interrupcion Boton 1 (P1.1)
150 #pragma vector=PORT1_VECTOR
151 __interrupt void Interrupcion_P1(void){
152     if(!(P1IN & BIT1)){
153         boton1 = 1;
154     }
155     P1IFG &= ~BIT1;
156     LPM0_EXIT;
157 }
158 
159 // Buzzer desactivado por defecto
160 volatile unsigned char buzzer_activo = 0;
161 
162 // Funcion que inicializa el buzzer
163 void init_buzzer(void) {
164     P2DIR  |= BIT6;
165     P2SEL  &= ~BIT6;
166     P2SEL2 &= ~BIT6;
167     P2OUT  &= ~BIT6;
```

```
168
169     TA1CTL = MC_0;
170     TA1CCTL0 = 0;
171     TA1CCR0 = 1000;
172 }
173
174 // Funcion que activa el sonido
175 void suena_hz(unsigned int hz){
176     unsigned long cuenta_media = 1000000UL / (unsigned long)hz; // 2MHz/(2*hz)
177
178     if(cuenta_media < 10) cuenta_media = 10;
179     if(cuenta_media > 65535) cuenta_media = 65535;
180
181     TA1CTL = TASSEL_2 | ID_3 | MC_1;  // SMCLK/8 => 2MHz, UP
182     TA1CCR0 = (unsigned int)(cuenta_media - 1);
183     TA1CCTL0 = CCIE;
184
185     buzzer_activo = 1;
186 }
187
188 // Funcion que desactiva el sonido
189 void apaga_sonido(void){
190     buzzer_activo = 0;
191     TA1CCTL0 &= ~CCIE;
192     TA1CTL = MC_0;
193     P2OUT &= ~BIT6;
194 }
195
196 // Rutina de interrupcion buzzer
197 #pragma vector=TIMER1_A0_VECTOR
198 __interrupt void Interrupcion_T1(void){
199     if(buzzer_activo) P2OUT ^= BIT6;
200     else              P2OUT &= ~BIT6;
201 }
202
203 // Funcion que segun el color que elijamos con el joystick, sonara un sonido u otro
204 void sonido_color(unsigned char color){
205     // 1: arriba, 2: derecha, 3: abajo, 4: izquierda
206     switch(color){
207     case 1: suena_hz(DO);  break;
208     case 2: suena_hz(MI);  break;
209     case 3: suena_hz(SOL); break;
210     case 4: suena_hz(SI);  break;
211     default: apaga_sonido();  break;
212     }
213 }
214
215 // LFSR (generador de secuencia pseudoaleatoria)
216 static uint16_t lfsr = 0xACE1u;
217
218 // Funcion para almacenar la semilla
219 void semilla(uint16_t s){
220     if(s == 0) {
221         s = 0xACE1u; // Evitamos semilla a 0
222     }
223     lfsr = s; // Guardamos la semilla
224 }
225
226 // Funcion para calcular los bits de la semilla
227 uint16_t lfsr_siguiente(void){
228     // Calculamos un nuevo bit como el XOR de varios taps
229     uint16_t bit = ((lfsr >> 0) ^ (lfsr >> 2) ^ (lfsr >> 3) ^ (lfsr >> 5)) & 1;
230     lfsr = (lfsr >> 1) | (bit << 15); // Desplazamos el bit y lo metemos por la izquierda
```

```
    return lfsr; // Devolvemos el nuevo valor
}

unsigned char aleatorio_1_4(void){
    return (unsigned char)((lfsr_siguiente() & 0x03) + 1);
}

// Funcion para dibujar una estrella simple (pequena o grande)
void dibuja_estrella(unsigned char x, unsigned char y, unsigned char grande, uint32_t color){
    Graphics_setForegroundColor(&g_sContext, color);

    if(!grande){
        Graphics_Rectangle p = (Graphics_Rectangle){x, y, x, y};
        Graphics_fillRectangle(&g_sContext, &p);
    }
    else{
        Graphics_Rectangle h = (Graphics_Rectangle){(int)(x-2), (int)y, (int)(x+2), (int)y};
        Graphics_Rectangle v = (Graphics_Rectangle){(int)x, (int)(y-2), (int)x, (int)(y+2)};
        Graphics_fillRectangle(&g_sContext, &h);
        Graphics_fillRectangle(&g_sContext, &v);
    }
}

// Funcion principal
int main(void){
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog-timer

    Set_Clk(16); // Reloj de 16 MHz
    inicia_ADC(BIT0 | BIT3); // Joystick A0 y A3

    // Iniciamos lo correspondiente a la pantalla LCD
    Crystalfontz128x128_Init();
    Crystalfontz128x128_SetOrientation(LCD_ORIENTATION_UP);
    Graphics_initContext(&g_sContext, &g_sCrystalfontz128x128);
    Graphics_setFont(&g_sContext, &g_sFontCm16b);
    Graphics_setBackgroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
    Graphics_clearDisplay(&g_sContext);

    init_buzzer();  // Inicializamos el buzzer
    timer_tick();   // Inicializamos el timer
    boton_start();  // Inicializamos el boton de start
    boton_1();      // Inicializamos el boton 1

    __bis_SR_register(GIE);

    // Dibujo de los elementos del juego
    Graphics_Rectangle boton[4]; // 4 rectangulos del simon dice

    unsigned int xC = 64; // Centro X
    unsigned int yC = 70; // Centro Y
    unsigned int c  = 26; // Tamano del hueco central
    unsigned int t  = 30; // Grosor del anillo

    // Coordenadas del hueco central
    unsigned int x0 = xC - c/2;
    unsigned int x1 = xC + c/2;
    unsigned int y0 = yC - c/2;
    unsigned int y1 = yC + c/2;

    // Coordenadas del contorno exterior del anillo
    unsigned int xL = xC - (c/2 + t);
    unsigned int xR = xC + (c/2 + t);
    unsigned int yT = yC - (c/2 + t);
```

```
294     unsigned int yB = yC + (c/2 + t);
295 
296     unsigned int gJ = 3; // Espacio de separacion entre rectangulos del anillo
297 
298     // Coordenadas de los rectangulos
299     boton[0] = (Graphics_Rectangle){(int)(x0 + gJ),(int)yT,(int)(xR),(int)y0}; // Amarillo (
        arriba)
300     boton[1] = (Graphics_Rectangle){(int)x1,(int)(y0 + gJ),(int)xR,(int)yB}; // Azul (derecha)
301     boton[2] = (Graphics_Rectangle){(int)(xL),(int)y1,(int)(x1 - gJ),(int)yB}; // Verde (abajo)
302     boton[3] = (Graphics_Rectangle){(int)xL, (int)yT, (int)x0, (int)(y1 - gJ)}; // Rojo (
        izquierda)
303 
304     // Marco exterior
305     Graphics_Rectangle marco = (Graphics_Rectangle){2, 2, 125, 125};
306 
307     // Barra de porcentaje
308     Graphics_Rectangle barra = (Graphics_Rectangle){12, 8, 110, 18};
309     Graphics_Rectangle caja_pct = (Graphics_Rectangle){112, 8, 120, 18};
310 
311     // Animacion victoria
312     Graphics_Rectangle barra1 = (Graphics_Rectangle){44,90,52,114};
313     Graphics_Rectangle barra2 = (Graphics_Rectangle){58,90,66,114};
314     Graphics_Rectangle barra3 = (Graphics_Rectangle){72,90,80,114};
315     Graphics_Rectangle marco_eq = (Graphics_Rectangle){36,86,88,118};
316 
317     // Array colores (arriba, derecha, abajo, izquierda)
318     uint32_t color_alto[4], color_bajo[4];
319 
320     // Array de simbolos
321     static const int8_t simbolos[4][2] = { {'#',0}, {'@',0}, {'$',0}, {'%',0} };
322 
323     // Asignamos los colores al array de colores iluminados
324     color_alto[0] = GRAPHICS_COLOR_YELLOW;
325     color_alto[1] = GRAPHICS_COLOR_BLUE;
326     color_alto[2] = GRAPHICS_COLOR_BRIGHT_GREEN;
327     color_alto[3] = GRAPHICS_COLOR_RED;
328 
329     // Asignamos los colores al array de colores apagados
330     color_bajo[0] = GRAPHICS_COLOR_DARK_YELLOW;
331     color_bajo[1] = GRAPHICS_COLOR_DARK_BLUE;
332     color_bajo[2] = GRAPHICS_COLOR_DARK_GREEN;
333     color_bajo[3] = GRAPHICS_COLOR_DARK_RED;
334 
335     // Juego
336     unsigned char secuencia[32]; // array que almacenara las 32 rondas de juego
337 
338     unsigned int ronda = 1;      // Empezamos en la ronda 1
339     unsigned int puntuacion = 0; // La puntuacion inicial es 0
340 
341     // Definimos las variables de los tiempos
342     unsigned int tiempo_base = 1000; // ms
343     unsigned int tiempo = 1000;      // ms
344     unsigned char modo_rapido = 0;
345 
346     unsigned int T = (unsigned int)(tiempo / 25);
347     if(T < 2) T = 2;
348     unsigned int T4 = (unsigned int)(T / 4);
349     if(T4 < 1) T4 = 1;
350     unsigned int T2 = (unsigned int)(2 * T);
351 
352     // Defino los estados de la FSM
353     typedef enum {
354         BIENVENIDA=0, MENSAJE_RONDA, MAQUINA, TURNO_JUGADOR,
```

```
        VICTORIA, VICTORIA_FINAL, FIN
    } estados_t;

    estados_t estado = BIENVENIDA;

    // Variables auxiliares de la FSM
    unsigned char sub_maquina = 0;
    unsigned int paso_maquina = 0;
    unsigned int paso_jugador = 0;
    unsigned int tms = 0;
    unsigned char estado_joystick = 0;
    unsigned char seleccion = 0;
    unsigned int tflash = 0;

    // Variables victoria
    unsigned int t_vict = 0;
    unsigned int paso_vict = 0;
    unsigned char anim = 0;

    // Variables victoria final
    unsigned int t_victf = 0;
    unsigned int paso_victf = 0;
    unsigned char animf = 0;

    // Melodia victoria final
    #define N_VICTF 18
    static const unsigned int melodia_victf[N_VICTF] = {
        DO,MI,SOL,DO2,RE2,MI2,RE2,DO2,
        0,SOL,LA,SOL,MI,FA,MI,RE,DO,0
    };
    static const unsigned char duracion_victf[N_VICTF] = {
        4,4,4,6,4,4,4,6,
        3,4,4,4,4,4,4,4,8,8
    };

    // Melodia bienvenida
    #define N_BIENV 16
    static const unsigned int melodia_bienv[N_BIENV] = {
        DO,0,MI,0,SOL,0,DO2,0,
        LA,0,SOL,0,FA,0,MI,0
    };
    static const unsigned char duracion_bienv[N_BIENV] = {
        4,2,4,2,4,2,6,3,
        4,2,4,2,4,2,6,6
    };

    unsigned int t_bienv = 0;
    unsigned int paso_bienv = 0;

    // Estrellas victoria final
    #define N_ESTRELLAS 12
    unsigned char star_x[N_ESTRELLAS];
    unsigned char star_y[N_ESTRELLAS];
    unsigned char star_big[N_ESTRELLAS];
    unsigned char star_on[N_ESTRELLAS];

    // Bucle infinito
    while(1){
        LPM0; // Dormimos al micro

        // Entramos cada 25ms
        if(tick){
            tick = 0;
```

```
418             tms++;
419
420             if(boton1){
421                 boton1 = 0;
422                 apaga_sonido();
423                 estado = VICTORIA_FINAL;
424                 tms = 0;
425             }
426
427             switch(estado){
428             // Pantalla de bienvenida
429             case BIENVENIDA:
430                 if(tms == 1){
431                     apaga_sonido(); // Buzzer desactivado
432
433                     tiempo = tiempo_base;
434                     modo_rapido = 0;
435
436                     Graphics_clearDisplay(&g_sContext);
437                     Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
438                     Graphics_drawStringCentered(&g_sContext, "SIMON DICE", -1, 64, 50,
    TRANSPARENT_TEXT);
439                     Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 75,
    TRANSPARENT_TEXT);
440
441                     t_bienv = 0;
442                     paso_bienv = 0;
443                 }
444
445                 t_bienv++;
446
447                 if(t_bienv == 1){
448                     if(melodia_bienv[paso_bienv] == 0){
449                         apaga_sonido();
450                     }
451                     else{
452                         suena_hz(melodia_bienv[paso_bienv]);
453                     }
454                 }
455
456                 if(t_bienv >= duracion_bienv[paso_bienv]){
457                     t_bienv = 0;
458                     paso_bienv++;
459                     if(paso_bienv >= N_BIENV){
460                         paso_bienv = 0;
461                     }
462                 }
463
464                 // Si la flag de start esta a 1, iniciamos el juego
465                 if(start){
466                     start = 0;
467                     apaga_sonido();
468
469                     // Modo accesible
470                     ejex = (unsigned int)lee_ch(0);
471                     if(ejex < 100) {
472                         modo_simbolos = 1; // Joystick a la izquierda
473                     }
474                     else {
475                         modo_simbolos = 0; // Centro/derecha colores
476                     }
477
478                     // Se decide la semilla aleatoria
```

```
                    semilla((uint16_t)(contador_ticks ^ 0xBEEF));
                    for(i=0;i<32;i++){
                        secuencia[i] = aleatorio_1_4();
                    }

                    // Reset del juego
                    ronda = 1;
                    puntuacion = 0;
                    paso_maquina = 0;
                    paso_jugador = 0;
                    sub_maquina = 0;

                    // Duracion de la secuencia de la maquina
                    T = (unsigned int)(tiempo / 25);
                    if(T < 2) T = 2;

                    // Tiempo entre pasos de la secuencia
                    T4 = (unsigned int)(T / 4);
                    if(T4 < 1) T4 = 1;

                    // Duracion de la secuencia del jugador
                    T2 = (unsigned int)(2 * T);

                    estado = MENSAJE_RONDA;
                    tms = 0;
                }
                break;

            // Estado de mensaje entre ronda y ronda
            case MENSAJE_RONDA:
                if(tms == 1){
                    char cad[20];
                    apaga_sonido(); // Buzzer desactivado

                    // Dibujamos el texto de por que ronda vamos
                    Graphics_clearDisplay(&g_sContext);
                    sprintf(cad, "RONDA %d", ronda);
                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                    Graphics_drawStringCentered(&g_sContext, (int8_t*)cad, -1, 64, 55,
TRANSPARENT_TEXT);
                    Graphics_drawStringCentered(&g_sContext, "MIRA...", -1, 64, 80,
TRANSPARENT_TEXT);

                    paso_maquina = 0;
                    sub_maquina = 0;
                }

                // Tiempo de espera la pantalla de ronda
                if(tms >= (800/25)){
                    Graphics_clearDisplay(&g_sContext); // Limpiamos la pantalla

                    // Dibujamos el marco
                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                    Graphics_drawRectangle(&g_sContext, &marco);

                    // Dibujamos la barra de progreso de la ronda
                    Graphics_drawRectangle(&g_sContext, &barra);
                    Graphics_drawRectangle(&g_sContext, &caja_pct);
                    Graphics_drawString(&g_sContext, "%", -1, 122, 7, TRANSPARENT_TEXT);

                    // Dibujamos los 4 rectangulos de color o simbolos
                    for(i=0;i<4;i++){
                        if(!modo_simbolos){
```

```
                            Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                        }
                        else{
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                            Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[i],-1,
                                (boton[i].xMin + boton[i].xMax)/2,
                                (boton[i].yMin + boton[i].yMax)/2,TRANSPARENT_TEXT);
                        }
                    }

                    estado = MAQUINA;
                    tms = 0;
                }
                break;

            case MAQUINA:
                // Si ya se han mostrado todos los pasos, pasamos al turno del jugador
                if(paso_maquina >= ronda){
                    apaga_sonido(); // Buzzer desactivado

                    // Redibujamos estado inactivo
                    for(i=0;i<4;i++){
                        if(!modo_simbolos){
                            Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                        }
                        else{
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                            Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[i],-1,
                                (boton[i].xMin + boton[i].xMax)/2,
                                (boton[i].yMin + boton[i].yMax)/2,TRANSPARENT_TEXT);
                        }
                    }

                    paso_jugador = 0;
                    estado_joystick = 0;
                    estado = TURNO_JUGADOR;
                    tms = 0;
                    break;
                }

                // Subestado para iluminar color
                if(sub_maquina == 0){
                    if(tms == 1){

                        unsigned char color = secuencia[paso_maquina];
                        unsigned char color_aux = color - 1; // Array de 0 a 3

                        if(!modo_simbolos){
                            Graphics_setForegroundColor(&g_sContext, color_alto[color_aux]);
                            Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
                        }
                        else{
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                            Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                            Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[color_aux
    ],-1,
```

```
                                (boton[color_aux].xMin + boton[color_aux].xMax)/2,
                                (boton[color_aux].yMin + boton[color_aux].yMax)/2,
    TRANSPARENT_TEXT);
                        }

                        sonido_color(color);

                        // Barra progreso maquina (relleno rojo)
                        Graphics_Rectangle interior = barra;
                        interior.xMin += 1; interior.yMin += 1;
                        interior.xMax -= 1; interior.yMax -= 1;

                        Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                        Graphics_fillRectangle(&g_sContext, &interior);

                        unsigned int W = interior.xMax - interior.xMin;
                        unsigned int relleno = (unsigned int)((unsigned long)W * (paso_maquina
    +1) / ronda);
                        if(relleno > 0){
                            Graphics_Rectangle f = interior;
                            f.xMax = f.xMin + relleno;
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_RED);
                            Graphics_fillRectangle(&g_sContext, &f);
                        }
                    }

                    // Tras T segundos
                    if(tms >= T){
                        unsigned char color = secuencia[paso_maquina];
                        unsigned char color_aux = color - 1;

                        if(!modo_simbolos){
                            Graphics_setForegroundColor(&g_sContext, color_bajo[color_aux]);
                            Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
                        }
                        else {
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                            Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                            Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[color_aux
    ],-1,
                                (boton[color_aux].xMin + boton[color_aux].xMax)/2,
                                (boton[color_aux].yMin + boton[color_aux].yMax)/2,
    TRANSPARENT_TEXT);
                        }

                        apaga_sonido(); // Buzzer desactivado
                        sub_maquina = 1; // Pasamos al subestado de pausa entre colores
                        tms = 0;
                    }
                }
                else {
                    if(tms >= T4){
                        paso_maquina++; // Siguiente paso
                        sub_maquina = 0;
                        tms = 0;
                    }
                }
                break;

            case TURNO_JUGADOR: {
                // Si se acaba el tiempo, game over
                if(tms >= T2){
```

```
                    estado = FIN;
                    tms = 0;
                    apaga_sonido();
                    break;
                }

                if(tms == 1){
                    // Inicializamos los botones apagados
                    for(i=0; i<4; i++){
                        if(modo_simbolos){
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                            Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                            Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[i],-1,
                                (boton[i].xMin + boton[i].xMax)/2,
                                (boton[i].yMin + boton[i].yMax)/2,TRANSPARENT_TEXT);
                        }
                        else{
                            Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
                            Graphics_fillRectangle(&g_sContext, &boton[i]);
                        }
                    }
                }

                // Leer joystick
                ejex = (unsigned int)lee_ch(0);
                ejey = (unsigned int)lee_ch(3);

                int dx = (int)ejex - 512;
                int dy = (int)ejey - 512;

                // Zona muerta
                if(dx < 100 && dx > -100 && dy < 100 && dy > -100){
                    estado_joystick = 0;
                }

                unsigned char elegido = 0;

                if(estado_joystick == 0){
                    if(!(dx < 100 && dx > -100 && dy < 100 && dy > -100)) {
                        estado_joystick = 1;
                        int adx = (dx<0)?-dx:dx;
                        int ady = (dy<0)?-dy:dy;

                        if(adx > ady) {
                            if(dx > 0) elegido = 2;  // Derecha
                            else       elegido = 4;  // Izquierda
                        }
                        else {
                            if(dy > 0) elegido = 1;  // Arriba
                            else       elegido = 3;  // Abajo
                        }
                    }
                }

                // Si se ha elegido algun color
                if(elegido != 0) {
                    unsigned char esperado = secuencia[paso_jugador];
                    {
                        unsigned char color_aux = elegido - 1;
                        if(!modo_simbolos) {
                            Graphics_setForegroundColor(&g_sContext, color_alto[color_aux]);
                            Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
```

```
724                         }
725                         else {
726                             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
727                             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
728                             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
729                             Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[color_aux
    ],-1,
730                                 (boton[color_aux].xMin + boton[color_aux].xMax)/2,
731                                 (boton[color_aux].yMin + boton[color_aux].yMax)/2,
    TRANSPARENT_TEXT);
732                         }
733
734                         sonido_color(elegido);
735                         seleccion = elegido;
736                         tflash = 0;
737                     }
738
739                     // Si el color elegido es el correcto
740                     if(elegido == esperado){
741                         puntuacion++;
742                         paso_jugador++;
743
744                         // Barra progreso jugador (relleno blanco)
745                         Graphics_Rectangle interior = barra;
746                         interior.xMin += 1; interior.yMin += 1;
747                         interior.xMax -= 1; interior.yMax -= 1;
748
749                         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
750                         Graphics_fillRectangle(&g_sContext, &interior);
751
752                         unsigned int W = interior.xMax - interior.xMin;
753                         unsigned int relleno = (unsigned int)((unsigned long)W * (paso_jugador)
    / ronda);
754                         if(relleno > 0){
755                             Graphics_Rectangle f = interior;
756                             f.xMax = f.xMin + relleno;
757                             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
758                             Graphics_fillRectangle(&g_sContext, &f);
759                         }
760
761                         // Si completa la ronda
762                         if(paso_jugador >= ronda){
763                             estado = VICTORIA;
764                             tms = 0;
765                             t_vict = 0;
766                             paso_vict = 0;
767                             anim = 0;
768                             apaga_sonido();
769                         }
770                         else{
771                             tms = 0;
772                         }
773                     }
774                     else {
775                         estado = FIN; // ERROR
776                         tms = 0;
777                     }
778                 }
779
780                 if(seleccion != 0){
781                     tflash++;
782                     if(tflash >= (T/2)) {
783                         unsigned char color_aux = seleccion - 1;
```

```
784                         if(!modo_simbolos) {
785                             Graphics_setForegroundColor(&g_sContext, color_bajo[color_aux]);
786                             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
787                         }
788                         else {
789                             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
790                             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
791                             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
792                             Graphics_drawStringCentered(&g_sContext,(int8_t*)simbolos[color_aux
    ],-1,
793                                 (boton[color_aux].xMin + boton[color_aux].xMax)/2,
794                                 (boton[color_aux].yMin + boton[color_aux].yMax)/2,
    TRANSPARENT_TEXT);
795                         }
796
797                         apaga_sonido();
798                         seleccion = 0;
799                     }
800                 }
801                 break;
802             }
803
804             // Estado de ronda completada
805             case VICTORIA:
806                 if(tms == 1) {
807                     Graphics_clearDisplay(&g_sContext);
808                     Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
809                     Graphics_drawStringCentered(&g_sContext, "RONDA", -1, 64, 45,
    TRANSPARENT_TEXT);
810                     Graphics_drawStringCentered(&g_sContext, "SUPERADA", -1, 64, 65,
    TRANSPARENT_TEXT);
811
812                     Graphics_drawRectangle(&g_sContext, &marco_eq);
813                     Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
814                     Graphics_fillRectangle(&g_sContext, &barra1);
815                     Graphics_fillRectangle(&g_sContext, &barra2);
816                     Graphics_fillRectangle(&g_sContext, &barra3);
817
818                     paso_vict = 0;
819                     t_vict = 0;
820                     anim = 0;
821                 }
822
823                 t_vict++;
824
825                 // Musica simple de victoria
826                 if(paso_vict == 0) suena_hz(DO);
827                 if(paso_vict == 1) apaga_sonido();
828                 if(paso_vict == 2) suena_hz(MI);
829                 if(paso_vict == 3) apaga_sonido();
830                 if(paso_vict == 4) suena_hz(SOL);
831                 if(paso_vict == 5) apaga_sonido();
832                 if(paso_vict == 6) suena_hz(DO2);
833                 if(paso_vict == 7) apaga_sonido();
834                 if(paso_vict == 8) suena_hz(SOL);
835                 if(paso_vict == 9) apaga_sonido();
836                 if(paso_vict == 10) suena_hz(DO2);
837                 if(paso_vict == 11) apaga_sonido();
838
839                 if((paso_vict % 2) == 0) {
840                     if(t_vict >= 6) { t_vict = 0; paso_vict++; }
841                 }
842                 else {
```

```
                    if(t_vict >= 2) { t_vict = 0; paso_vict++; }
                }

                // Animacion ecualizador
                if((tms % 2) == 0){
                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
                    Graphics_fillRectangle(&g_sContext, &barra1);
                    Graphics_fillRectangle(&g_sContext, &barra2);
                    Graphics_fillRectangle(&g_sContext, &barra3);

                    unsigned char h1=8, h2=18, h3=12;
                    if(anim==0) { h1=8;  h2=18; h3=12; }
                    if(anim==1) { h1=18; h2=10; h3=16; }
                    if(anim==2) { h1=12; h2=16; h3=8;  }
                    if(anim==3) { h1=16; h2=8;  h3=18; }

                    if((paso_vict % 2) == 1){ h1 = 6; h2 = 6; h3 = 6; }

                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                    Graphics_Rectangle b1 = barra1; b1.yMin = (unsigned int)(barra1.yMax - h1);
                    Graphics_Rectangle b2 = barra2; b2.yMin = (unsigned int)(barra2.yMax - h2);
                    Graphics_Rectangle b3 = barra3; b3.yMin = (unsigned int)(barra3.yMax - h3);

                    Graphics_fillRectangle(&g_sContext, &b1);
                    Graphics_fillRectangle(&g_sContext, &b2);
                    Graphics_fillRectangle(&g_sContext, &b3);

                    anim++;
                    if(anim >= 4) anim = 0;
                }

                // Fin animacion
                if(paso_vict >= 12) {
                    apaga_sonido();
                    ronda++;
                    if(ronda > 32) {
                        estado = VICTORIA_FINAL;
                        tms = 0;
                        break;
                    }
                    paso_maquina = 0;
                    sub_maquina = 0;
                    paso_jugador = 0;
                    seleccion = 0;
                    estado = MENSAJE_RONDA;
                    tms = 0;
                }
                break;

            // Estado de victoria final
            case VICTORIA_FINAL:
                if(tms == 1){
                    apaga_sonido();
                    Graphics_clearDisplay(&g_sContext);

                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                    Graphics_drawStringCentered(&g_sContext, "VICTORIA!", -1, 64, 45,
    TRANSPARENT_TEXT);
                    Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 95,
    TRANSPARENT_TEXT);

                    t_victf = 0;
                    paso_victf = 0;
```

```
                    animf = 0;

                    for(i=0;i<N_ESTRELLAS;i++){
                        star_x[i] = (unsigned char)(4 + (lfsr_siguiente() % 120));
                        if((lfsr_siguiente() & 1) == 0) star_y[i] = (unsigned char)(8 + (lfsr_siguiente() % 22));
                        else                            star_y[i] = (unsigned char)(85 + (lfsr_siguiente() % 35));
                        star_big[i] = (unsigned char)(lfsr_siguiente() & 1);
                        star_on[i] = 0;
                    }
                }

                t_victf++;

                if(t_victf == 1){
                    if(melodia_victf[paso_victf] == 0) apaga_sonido();
                    else suena_hz(melodia_victf[paso_victf]);
                }

                if(t_victf >= duracion_victf[paso_victf]){
                    t_victf = 0;
                    paso_victf++;
                    if(paso_victf >= N_VICTF) paso_victf = 0;
                }

                if((tms % 2) == 0){
                    unsigned char k = (unsigned char)(lfsr_siguiente() % N_ESTRELLAS);
                    if(star_on[k]){
                        dibuja_estrella(star_x[k], star_y[k], star_big[k], GRAPHICS_COLOR_BLACK);
                        star_on[k] = 0;
                    }else{
                        star_big[k] = (unsigned char)(star_big[k] ^ 1);
                        dibuja_estrella(star_x[k], star_y[k], star_big[k], GRAPHICS_COLOR_YELLOW);
                        star_on[k] = 1;
                    }
                    animf++;
                    if(animf >= 4) animf = 0;
                }

                if(start){
                    start = 0;
                    modo_rapido = 1; // Aumentamos velocidad
                    tiempo = (unsigned int)(tiempo_base/2);

                    T = (unsigned int)(tiempo / 25);
                    if(T < 2) T = 2;
                    T4 = (unsigned int)(T / 4);
                    if(T4 < 1) T4 = 1;
                    T2 = (unsigned int)(2 * T);

                    semilla((uint16_t)(contador_ticks ^ 0xBEEF));
                    for(i=0;i<32;i++) secuencia[i] = aleatorio_1_4();

                    ronda = 1; puntuacion = 0;
                    paso_maquina = 0; paso_jugador = 0;
                    sub_maquina = 0; seleccion = 0;

                    estado = MENSAJE_RONDA;
                    tms = 0;
                    apaga_sonido();
```

```
                }
                break;

            // Estado de game over
            case FIN:
                if(tms == 1) {
                    char cad[24];
                    apaga_sonido();

                    if(modo_rapido){
                        modo_rapido = 0;
                        tiempo = tiempo_base;
                        T = (unsigned int)(tiempo / 25); if(T < 2) T = 2;
                        T4 = (unsigned int)(T / 4); if(T4 < 1) T4 = 1;
                        T2 = (unsigned int)(2 * T);
                    }

                    Graphics_clearDisplay(&g_sContext);
                    Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
                    Graphics_drawStringCentered(&g_sContext, "GAME OVER", -1, 64, 45,
    TRANSPARENT_TEXT);
                    sprintf(cad, "PUNTOS: %d", puntuacion);
                    Graphics_drawStringCentered(&g_sContext, (int8_t*)cad, -1, 64, 70,
    TRANSPARENT_TEXT);
                    Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 95,
    TRANSPARENT_TEXT);

                    suena_hz(150); // Sonido derrota
                }
                if(tms >= (600/25)) apaga_sonido();

                if(start) {
                    start = 0;
                    estado = BIENVENIDA;
                    tms = 0;
                }
                break;
            }
        }
    }
}
```

Listing 1: Complete code for the Simon Says game (main.c)